

**PAM4 8x53Gpbs with QSFPDD Serial loopback
design on the Nios[®] V/m Processor**

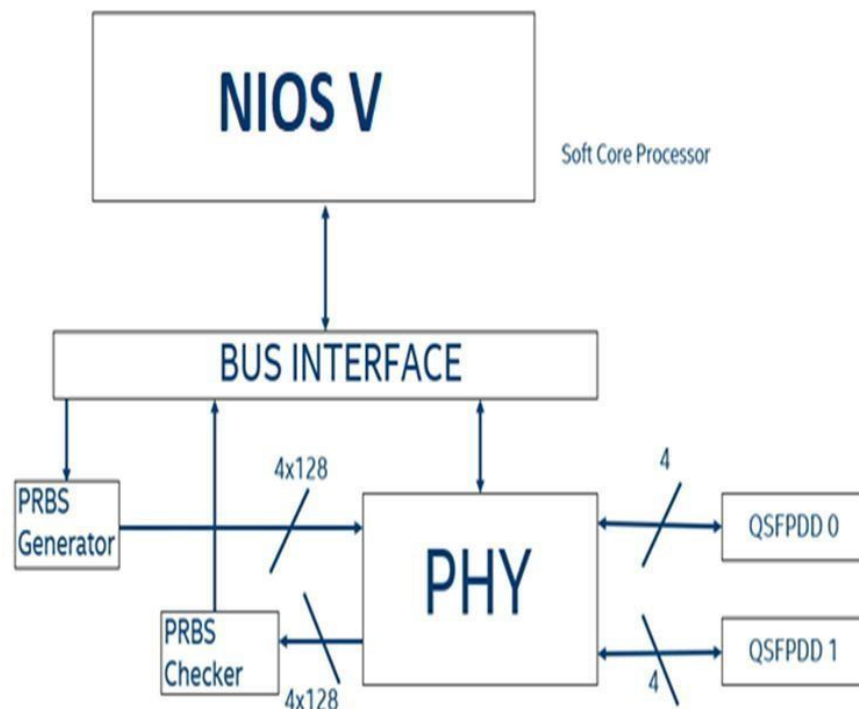
Agilex[™] 7 FPGA

Contents

1. Theory of Operation	3
1.1. IP Cores	4
2. Directory Structure	5
3. Quick Start Guide	5
4. Generating the Design	6
4.1. Building Hardware Design	6
4.2. Building Software Design	6
5. Programming the Design	7
6. Expected Results	8

1. Theory of Operation

Figure 1-1. Block Diagram



- Uses Agilex™ 7 FPGA F-Series Development Kit (2x F-Tile) as platform for demonstration.
- Uses 2 Native PHY's with each 4 independent physical lanes connected to QSFPDD0 and QSFPDD1 and each channel configured in RSFEC53Gbps PAM4 using 8 FGT's located in Quad 3 and Quad 1
- As there is only one clock source on the board both phy's are clocked from the same reference clock.
- Uses dedicated txrx_pcs_64b66b_fgt module which instantiates all relevant additional logic required for RSFEC operation.
- User data width is 128-bits per physical lane.
- Since RSFEC is used in the PHY Direct IP, system PLL clocking is used. And both Tx and Rx user interface are clocked by the System PLL/2 clock which is running at $53.125\text{Gbps}/128 = 415.0390625\text{ Mhz}$.
- Each channel is fully independent and can be independently reset without impacting the other channels.
- Uses Release Reset IP
- Design uses a Qsys system with Nios® V/m processor to control the PHY Direct IP's used in the PHY and control all other logic (like temperature measurement etc.)

- To create the 2x4 128 bit user data 128-bit soft PRBS generators and 128-bit PRBS checkers are being used. Supported PRBS patterns : PRBS-7, PRBS-13, PRBS-23, PRBS-31.
- Each 128-bit PRBS Generator will use a different seed to avoid the same pattern being sent on the 512 bit bus going to each PHY.
- Uses Scrambler/Descrambler on top of the PRBS pattern as a scrambler is required on user data in combination with the FEC. Even though mandatory for FEC the scrambler can be dynamically enabled/disabled. Each virtual lane (64 bit) has it's own scrambler/descrambler. Scramblers/Descramblers used are self-synchronizing as used by Ethernet IEEE interfaces with polynomial $1 + x^{39} + x^{58}$
- Supports serial loopback
- Additional control signals required for the RSFEC operation are also generated in the design in module txrx_pcs_64b66b_fgt
- Used 2 different AVMM interfaces to the PHY direct IP: the transceiver reconfiguration interface and the transceiver datapath interface.
- Reports errorcount and calculates BER based on the received data.
- Extensive FEC statistics are provided for each virtual lane and physical lane
- Displays measured core temperatures.

1.1. IP Cores

The following IPs are used in this design.

- Nios V/m soft processor core
- On Chip RAM-II
- JTAG UART
- Sys ID
- F-Tile PMA/FEC Direct PHY Intel FPGA IP
- F-Tile Reference and System PLL Clocks Intel FPGA IP

2. Directory Structure

The directory structure is explained below:

Root directory: `agf027files-dev-devkit/niosv_m/xcvr_ser_lp`

Directories	Content
docs	Document capturing the details of the example design
img	Block diagram for the example designs
ready_to_test	Prebuilt binaries (FPGA <code>.sof</code> file and Application <code>.elf</code> file) which can be programmed on the board and tested
sources - hw - scripts - sw/app	Files needed to create the design - Necessary hardware files (<code>.sv</code>, <code>.v</code>, <code>.ip</code>) of the design - This folder consists of scripts to build the design - This folder contains software application files

3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA `sof` and Application `elf` files are found in `ready_to_test` folder.

Refer to [Chapter 6 Programming the Design](#) for the steps to program these files.

4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the `.sof` file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory:
`> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application `.elf` file into `.hex` file for memory initialization.

To create software application, run the following commands in the terminal:

1. `niosv-bsp -c --quartus-project=hw/devkit_demo.qpf --qsys=hw/controller.qsys --type=hal sw/bsp/hal_default/settings.bsp`
2. `niosv-app --bsp-dir=sw/bsp/hal_default --app-dir=sw/app/Agilex_xCh --elf-name=Agilex_xCh.elf --srcs=sw/app/Agilex_xCh/input_functions.c --srcs=sw/app/Agilex_xCh/channel_functions.c --srcs=sw/app/Agilex_xCh/nphy_functions.c --srcs=sw/app/Agilex_xCh/PMA_functions_FTILE.c --srcs=sw/app/Agilex_xCh/FEC_functions_FTILE.c --srcs=sw/app/Agilex_xCh/main.c`
3. `cmake -S sw/app/Agilex_xCh -B sw/app/Agilex_xCh`
4. `make -C sw/app/Agilex_xCh`

Note: It is recommended to clean the `app/build` project before regenerating `elf` (cmake and make).

5. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling* RiscFree* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated `sof`, then download the `elf` file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;devkit_demo.sof'

#----- Download the elf file on the board -----#
> niosv-download -g ready_to_test/Agilex_xCh.elf -c 1

#----- Verify the output on the terminal by using the following command in the
terminal -----#
> juart-terminal -d 1 -c 1 -i 0
```

Note: Reduce the JTAG clock frequency to 6MHz before programming the application `.elf` file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

6. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

```
01;32mPASS : All channels are locked and errorfree01m

Select Action :
=====
P. Toggle Display to show info for All Phy's or only Selected Phy
.. Set all channels in serial loopback except selected channel
[. Mute all transmitters
]. Unmute all transmitters
(. Reset all channels except selected channel
). De-assert Reset on all channels
1. Perform a reset on the Selected Phy
~. Perform a reset on the Selected channel
$. Perform a rx_reset on the Selected channel
^. Perform a tx_reset on the Selected channel
2. Select Phy
3. Select Channel To Control
4. Toggle between Prbs and Scrambled Idle Pattern on Selected Phy
5. Insert Soft Prbs Biterror on Selected Channel
*. Inject RSFEC errors during 1ms on Selected Channel/Segment in Selected PHY (pat=0x01,rate=0x01)
#. Build FEC Errortree live for Selected Phy
6. Reset ErrorCounter on selected Channel
7. Show Status
8. Control Serial loopback in Selected Phy
{. Control Reverse Parallel loopback in Selected Phy
9. Reset Errorcount on all channels in Selected Phy
A. Select Prbs Pattern on Selected Channel
?. Enable/Disable Scrambling of the datapath on all Phys
C. continuously update BER and errorcount every 2 seconds
D. Input new BER Time Interval
E. Show Transceiver PMA Settings on all channels
G. Change TX PMA settings on the Selected Phy
M. Control Tx Polarity in Selected Phy
N. Control Rx Polarity in Selected Phy
O. Perform Vertical Eye Height Measurement on selected Physical Channel
Q. Clear Errorcounters on all Phys (including RSFEC counters)
R. Rerun adaptation on all channels in Selected Phy
U. Sweep PMA settings and program best setting on selected channel
!. Read Register of PHY Direct in Selected Phy using rd_channel
.. Read Register of PHY Direct in Selected Phy using rd_channel_ftile
L. Stress test functions
Z. Dump address space

juart-terminal: exiting due to ^D on remote
01;32mPASS : All channels are locked and errorfree01m
```